



Algeria Pilot — Cohort 01

Data Science Internship *Capstone Presentation*

Car Price Prediction & Iris Flower Classification

PRESENTED

Saaidia Amina

DOMAIN

Data Science

SUPERVISOR

Thamer Niati

Presentation Agenda

01



Presenter Background

Transition from Refining & Process Engineering to Data Science applications.

02



Deliverables & Toolkit

Overview of Task 1, Task 3 objectives, and shared technology toolkit.

03



Task 3: Car Price Prediction

End-to-end regression pipeline, EDA drivers, and data leakage correction.

04



Task 1: Iris Classification

Multi-class classification workflow, feature scaling, and precision evaluation.

05



Working Through Obstacles

Technical solutions for data leakage, silent tie-breaking bugs, and EDA sync.

06



Submission & Wrap-up

GitHub repository deliverables, capstone completion, and discussion.

From Process Engineering to Data Science



Academic Foundation

Master 1 — Refining & Process Engineering

Engineering degree focused on **petroleum refining**, with coursework spanning refining operations, static equipment (columns, vessels, heat exchangers), gas treatment, and industrial process optimization.

Working with process variables, yield calculations, and equipment performance data built the same instincts data science relies on: cleaning noisy measurements, spotting the variables that actually drive an outcome, and validating a result before trusting it.



Data Science in Refining

A refinery runs on measurement — temperatures, flow rates, pressures, and equipment readings captured continuously across every unit. Data science doesn't replace that process engineering knowledge; it extends it, using those same numbers to catch patterns a person monitoring dashboards in real time would miss. In practice, that means:

- **Predictive maintenance** — catching early signs of equipment wear before they cause downtime
- **Process & yield optimization** — using historical data to find the conditions that actually drive better yield
- **Quality & anomaly detection** — flagging off-spec product or abnormal readings the moment they happen
- **Data-driven decisions** — backing choices with what the plant's data shows, not just habit

Task Overviews & Objectives



Task 3

Car Price Prediction

Regression problem: predict the resale price of used cars from the **CarDekho dataset** (Indian used-car market, ~8,128 listings).

OBJECTIVE

Build an accurate, honestly-validated regression pipeline that generalizes to unseen listings.

- **Numeric target:** selling price (INR)
- **End-to-end pipeline:** **cleaning** → **features** → **train** → **predict** → **evaluate**
- **Best model:** Gradient Boosting Regressor



Task 1

Iris Flower Classification

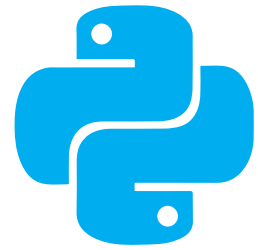
Multi-class classification problem: identify the species of an iris flower from four measured features.

OBJECTIVE

Compare several classifiers on a clean benchmark dataset and select the strongest performer.

- **Three classes:** Setosa, Versicolor, Virginica
- **Models compared:** Logistic Regression, KNN, Random Forest
- **Best model:** K-Nearest Neighbors (KNN)

Technologies & Tools Used



Python

Core programming language for the full workflow



Jupyter Notebooks

Exploratory, documented analysis in .ipynb files



Pandas & NumPy

Data loading, cleaning, and numeric computation



Scikit-learn

Pipelines, preprocessing, and model training



Matplotlib & Seaborn

Exploratory data analysis and visualization



Git & GitHub

Version control and public repository submission

Steps Passed Through — Cleaning & Features



01

Data Cleaning

The raw data had numbers mixed with text, which no model can use directly — so each field was cleaned up:

- "23.4 kmpl" → **mileage_kmpl** — just the number, unit removed
- "1248 CC" → **engine_cc** — engine size pulled out as a plain number
- "74 bhp" → **max_power_bhp** — power value cleaned the same way
- Torque field removed — mixed units made it too messy to trust



02

Feature Engineering

Created new columns that describe the car better than the raw data did:

- **car_age** calculated from the manufacturing year — easier for the model to use than a year on its own
- **Owner history** ("First Owner," "Second Owner," etc.) turned into **owner_num**, a simple number scale
- **Fuel type**, transmission, and seller type prepared so the model can read them
- Checked which details (like **brand**) were actually useful and which just added noise



03

Imputation & Outlier Handling

Data split into training and test sets first — then every fix below applied only to the training side :

- Missing values filled in using the **median**, since it isn't thrown off by extreme values
- Very high or low prices and **mileage** were capped instead of deleted, so no data was wasted
- All these fixes were done after splitting the data into **train** and **test** — fixing the leakage problem from before
- Double-checked afterward to make sure train and test data no longer **overlapped**

Steps Passed Through — EDA, Modeling & Evaluation



Exploratory Data Analysis (EDA)

- **Price Distribution:** *selling_price* showed a heavy right-skew, making log-transformation helpful to stabilize variance for linear models.
- **Top Price Drivers:** *max_power_bhp* and *engine_cc* showed the strongest positive connection to price—higher horsepower consistently matched higher listing prices.
- **Car Age Effect:** *car_age* had a strong negative trend, showing that cars lose most of their value quickly in the first 3 to 5 years.
- **Category Trends:** Boxplot comparisons showed that Automatic transmission and Diesel models consistently commanded higher median prices than Manual and Petrol ones.



Modeling & Evaluation

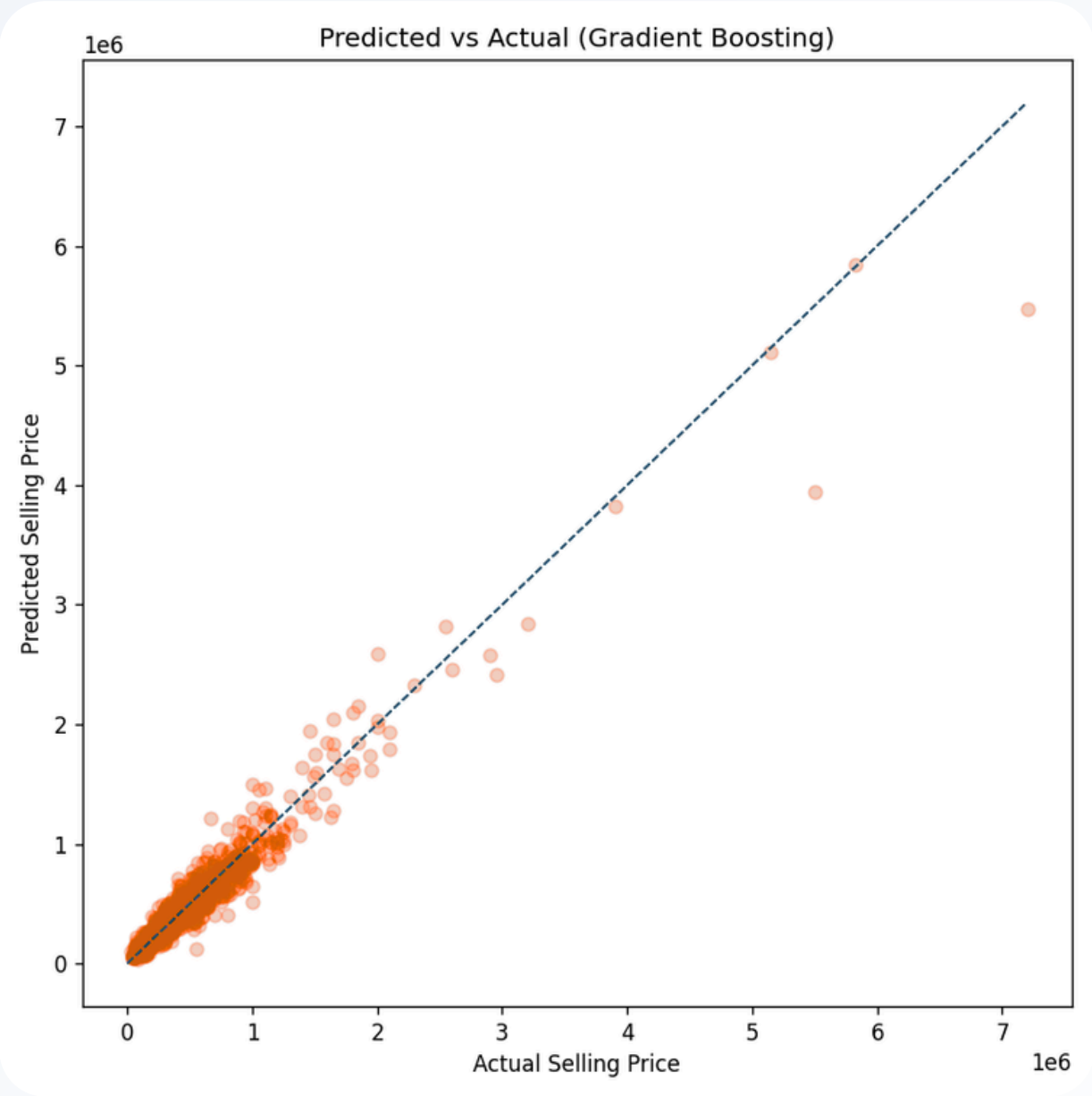
- All the cleaning and model steps chained together into one pipeline
- Several models trained on the training data
- Each trained model used to predict prices for cars it hadn't seen before (the test set)
- Predictions checked against the real prices using R^2 , RMSE, and MAE
- Gradient Boosting came out as the best-performing model

Model Comparison — Car Price Prediction

Three regression models were trained and evaluated on the same test set:

Model	RMSE	MAE	R ²
Gradient Boosting BEST MODEL	117,695	72,035	0.937
Random Forest	127,043	72,182	0.926
Linear Regression	300,052	168,911	0.590

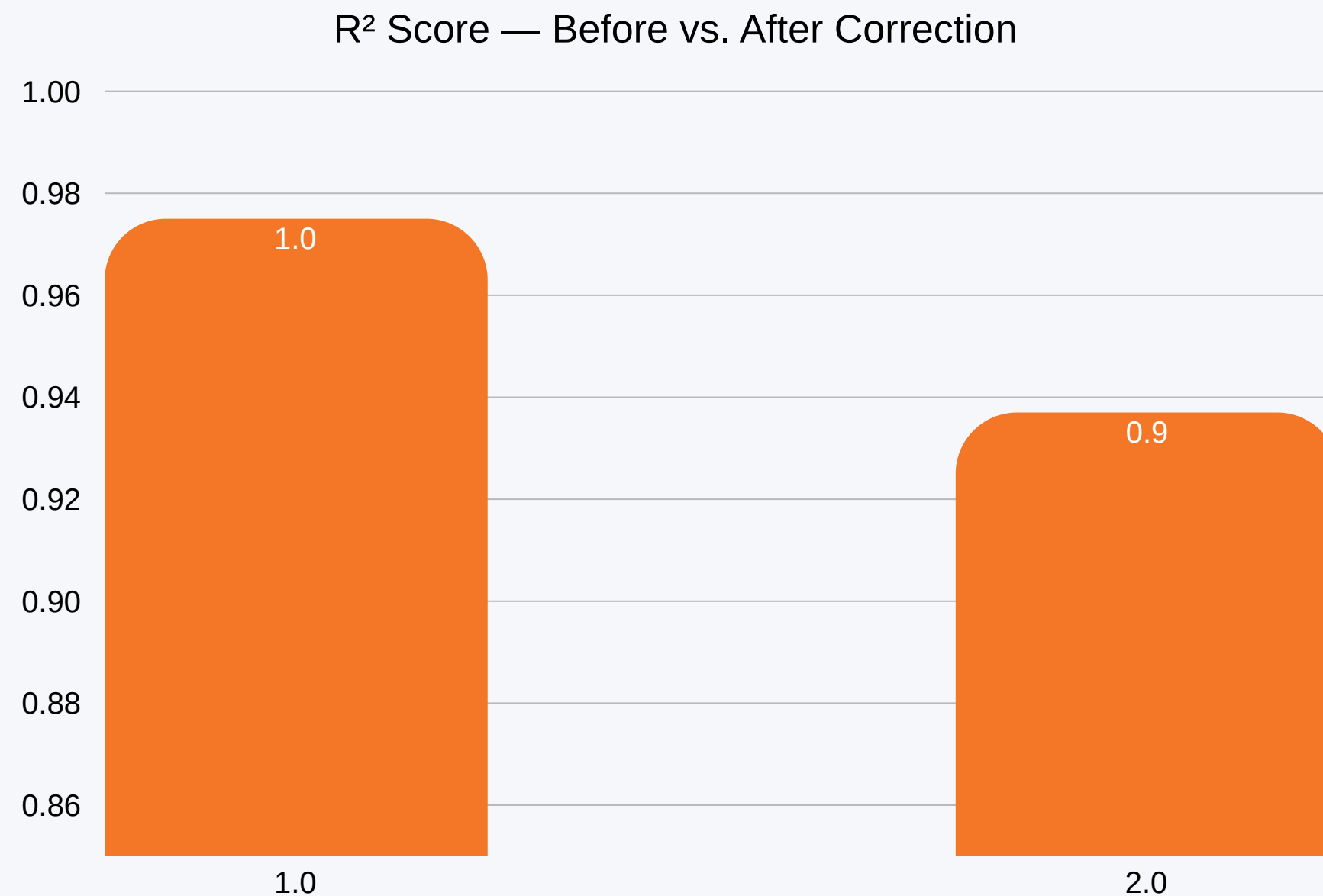
Note: Lower **RMSE** and **MAE** = smaller average error. Higher **R²** = more of the price variation explained. Gradient Boosting had the lowest error and the highest **R²**, so it was chosen as the final model.



- Each dot = one car from the test set. The dashed green line marks perfect prediction (predicted = actual).
- Dots closer to the line mean more accurate predictions.

Results — Catching a Data Leakage Issue

An early R^2 of 0.975 looked excellent — until closer review traced it to duplicate rows shared between train and test, and to imputation performed before the split. Both were fixed and re-validated.



Why the Lower Score Is the Right One

- 0.937 reflects genuine performance on truly unseen data
- Removing duplicates closed a direct train/test overlap
- Moving imputation after the split stopped test-set information from leaking into training
- Final pipeline: Gradient Boosting Regressor, validated with R^2 , RMSE, and MAE

Steps Passed Through



Load & Explore

Reviewed feature distributions across 150 balanced samples — found Setosa fully separable by petal measurements, and a strong 0.96 correlation between petal length and width

Prepare Data

Split features and labels, then applied StandardScaler — justified by the feature-scale differences found during EDA

Train Models

Trained and tuned Logistic Regression, **K-Nearest Neighbors**, and Random Forest classifiers

Predict & Evaluate

Used each trained model to predict species on the test set, measured accuracy, precision, recall, and F1 — KNN (K=5) led on precision

Key Engineering Insight:

Distance-based algorithms like KNN are highly sensitive to measurement scales. Applying StandardScaler ensured that larger sepal values did not unfairly dominate smaller petal dimensions during distance calculations.

Model Comparison — Iris Classification

Three regression models were trained and evaluated on the same test set:

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1 Score (Macro)
Logistic Regression	0.9333	0.9333	0.9333	0.9333
Random Forest	0.9333	0.9333	0.9333	0.9333
KNN (K=5) BEST MODEL	0.9333	0.9444	0.9333	0.9327

Note: KNN (K=5) was selected and saved as best_model.pkl because it achieved the highest Macro Precision (0.9444); choosing K=5 balanced local noise reduction with sharp boundary definition while preventing voting ties.



All models achieved 0.9333 accuracy, reflecting EDA findings where Setosa is linearly separable and errors occur only in the Versicolor–Virginica overlap.

Difficulties & Challenges Encountered



Data Leakage from Improper Splitting

TASK 3

My first model looked almost too good, with an R^2 of 0.975. Turns out some rows were duplicated across my train and test sets, and I was filling in missing values before splitting the data — so the model was quietly "seeing" test answers early. I fixed both issues and the real score came out to 0.937, which is the honest number.



A Silent Tie-Breaking Bug

TASK 1

My code picked the "best" model by checking if its accuracy was higher than the current best. But when all three models got the exact same accuracy, that check never triggered — so the code just kept the first model it saw, not the actual best one. I found this by comparing my terminal output to what I expected, then fixed it by comparing precision instead.



Keeping EDA and Training Data in Sync

TASK 1

My EDA notebook was looking at the raw dataset, but my model was actually trained on a cleaned version with duplicates removed. That mismatch caused wrong row counts and a missing-column error. I fixed it by making sure the EDA always uses the exact same cleaned data as the model.

SUBMISSION

Submission Deliverables

separate repository submitted for both tasks under the EXPS Nexus program



TASK 1 · CLASSIFICATION

Iris Flower Classification

<https://github.com/amina-saaidia/iris-classification.git>

KEY INCLUDED FILES

-  `notebooks/EDA.ipynb` — Data exploration & visual analysis
-  `src/` — Pipeline scripts (`data_prep.py` `train.py`)
-  `models/best_model.pkl` — Trained KNN classifier artifact
-  `app.py` — Interactive Streamlit web interface
-  `presentation/` & `README.md` — Capstone slides & setup guide



TASK 3 · REGRESSION

Car Price Prediction

https://github.com/amina-saaidia/Car_Price_Prediction.git

KEY INCLUDED FILES

-  `notebooks/EDA.ipynb` — Target skewness & price driver analysis
-  `src/` — Parsing, leakage-free prep & training scripts
-  `models/best_model.pkl` — Gradient Boosting Regressor
-  `app.py` — Real-time price prediction web app
-  `presentation/` & `README.md` — Deck slides & documentation



EXPERT PETROLEUM SERVICES **EXPS NEXUS**
PILOT — ALGERIA COHORT 01

Thank You

Questions & Discussion